Task 9: LangGraph + RAG Judge

Rami Al Habash 3/17/2026
CMPM 118 Dr. Leilani Gilpin

Goal

1. Reimplement LINC using LangGraph
2. Add a RAG relevance judge to the graph
3. Add an alternative route for the graph

Approach

I wanted to make my RAG retrieval more relevant. I used my Prolog file and an LLM to generate ProofWriter and FOLIO data. Afterwards, I employed RAG retrieval on the FOLIO data, then worked on creating the following LangGraph pipeline that does this for every query:

Retrieve FOLIO → judge relevance → if relevant → LINC → result
|
else → COT → result

I call this the "myLINC" pipeline. Refer to the README.md file to see the code and results:

LangGraph

I am a big fan of LangGraph. Implementing steps as nodes largely simplifies the coding process and breaks it down into clearer parts. I learned how to construct graphs and how to connect nodes. I also learned how to create routing nodes (conditional nodes). I used my code from task 8 and referred to the diagram above to create an appropriate pipeline that accurately responds to queries and runs benchmark tests.

Pipeline Explanation and LangSmith

How the pipeline works is whenever you give the LLM an example and ask it to determine whether a query is true, false, or uncertain, RAG retrieves data from the FOLIO dataset, which shows the LLM how to translate natural language into FOL. The relevance judge node examines if the retrieved FOLIO examples are enough to help the LLM translate the example you gave it to FOL. If the judge decides it's enough, then it will go through the traditional LINC pipeline (FOL translations and theorem provers). If it's not enough, it will use a COT approach.

I also learned how to use LangSmith/the LangSmith Observatory to get a better visual of the graph we created. I created an API key from the website and imported it into my coding environment. I was able to get a view of what each node outputs and what it goes through. I also saw if things ran successfully.

Takeaway

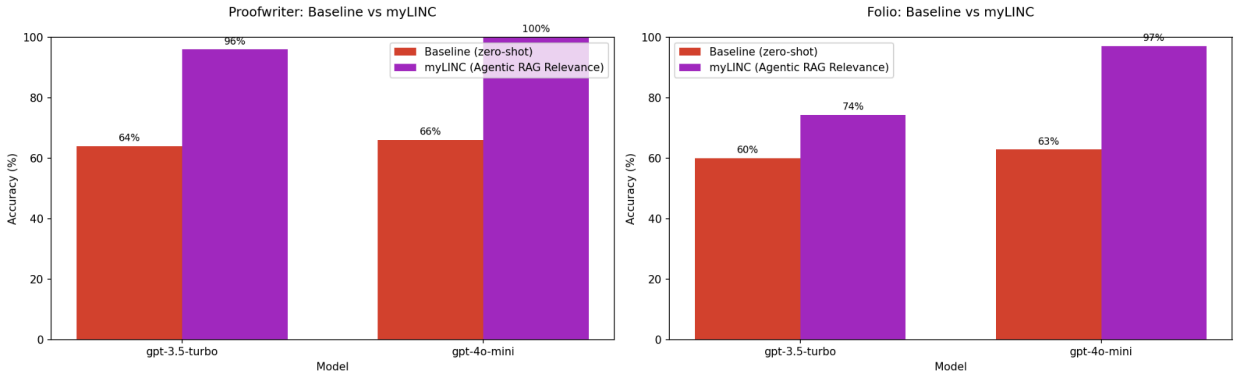
LangChain and LangGraph extremely simplify the coding process. They make it easy to call LLMs and create pipelines, respectively. I will be using them for my personal coding workflows.

Results and Analysis

Please view “results.txt” to see the results of running make run_queries:

- These results show that the LLM judge always routed to the traditional LINC route. This is expected because we are now using RAG on the FOLIO dataset, which has relevant translations.

Benchmark test:



These results are much better than task 8, showing improvement with using RAG on FOLIO.

References

Papers

LINC's study: https://arxiv.org/abs/2310.15164
COT: https://arxiv.org/abs/2201.11903
FOLIO: https://arxiv.org/abs/2209.00840
ProofWriter: https://arxiv.org/abs/2012.13048

AI-Assisted Coding

Used Claude to:

- Help me find and understand LangGraph functions.
- Assist with debugging where appropriate

I critically evaluated all outputs. I wrote all the code or got it from LINC/COT and made changes. I attributed LINC/COT where appropriate.